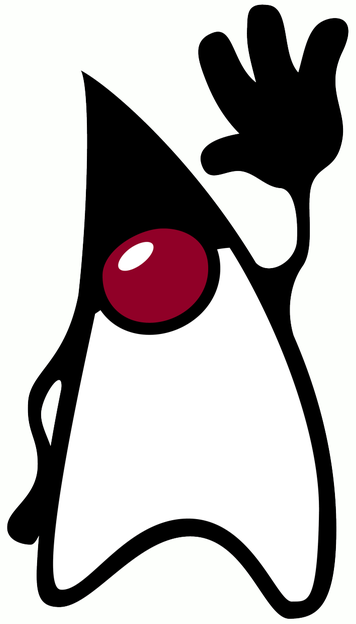


UNIVERSIDADE
FEDERAL DO CEARÁ
CAMPUS QUIXADÁ

Classes abstratas

QXD0007 - Programação Orientada a Objetos



Prof. Bruno Góis Mateus (brunomateus@ufc.br)

Conteúdo

- Introdução
- Classe abstrata
- Capturando um Pokemon

Introdução



Introdução

- Desde os primórdios do desenvolvimento de software, o conceito de reuso foi reinventado várias vezes
- O paradigma OO nos oferece diversos mecanismo que facilitam o reuso
- Contudo, a utilidade e a reutilização do código dependem de quão bem ele foi projetado e implementado
- Uma parte software particularmente importante no contexto de reuso é o que nós chamamos de contrato



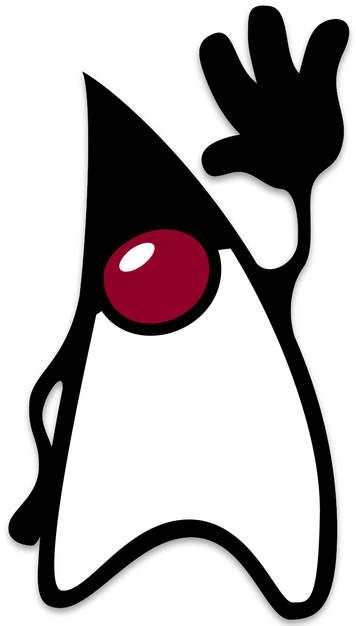
O termo **contrato** é amplamente usado em muitos aspectos dos negócios, incluindo desenvolvimento de software. Não confunda o conceito apresentado aqui com outros possíveis conceitos de **design de software**.

Introdução

- Contrato
  - Um mecanismo que exige que o desenvolvedor cumpra especificações
  - Isso inclui seguir nomes de métodos, número de parâmetros e outra coisas
  - Padrões criados para facilitar as boas práticas de desenvolvimento

Introdução

- Um desenvolvedor é sempre capaz de quebrar o contrato
- Um desenvolvedor iniciante pode resolver reinventar a roda ao invés de utilizar o especificação
- Um padrão deixa se ser benéfico caso ele não seja utilizado
- Garantir o cumprimento do contrato é vital



Em Java temos duas opções de definir esses contratos: Classes Abstratas e Interfaces

Classes abstratas



Classes abstratas

O que é?

- Representa uma ideia genérica demais para existir sozinha
  - Ex: Forma Geométrica
- Existe para ser herdada, nunca para ser instanciada
- Possibilitam herança de código preservando comportamento
- A classe abstrata declara o que toda subclasse deve saber fazer

Classes abstratas

Métodos abstratos

- É um métodos sem implementação
- Declaram o que toda subclasse deve saber fazer
- São implementados nas subclasses
  - É uma obrigação verificada pelo compilador
- Não dizem como: cada subclasse faz do seu jeito

Classes abstratas

Contrato com implementação parcial

- Métodos abstratos são a parte do contrato que cada subclasse preenche
- A parte já entregue pronta
  - Podem ter atributos, construtor e métodos concretos
    - Métodos concretos são métodos com implementação
    - Podem ter construtores, para reuso
- A classe abstrata é um contrato que já vem com parte do trabalho feito

Classes abstratas

As regras em Java

- Não pode ser instanciada com new
- Método abstrato não tem corpo
- Tem método abstrato? A classe precisa ser abstrata
- Subclasse concreta implementa todos os métodos abstratos, ou também vira abstrata

Classes abstratas

Quando utilizar?

- Existe uma relação verdadeira de “é um”
- Quando faz sentido proibir objetos da classe genérica
  - Faz sentido existir um objeto dessa classe sozinho?
- Quando há comportamento base comum entre subclasses
  - Evitar duplicação de código
- Quando você precisa garantir a implementação de métodos específicos

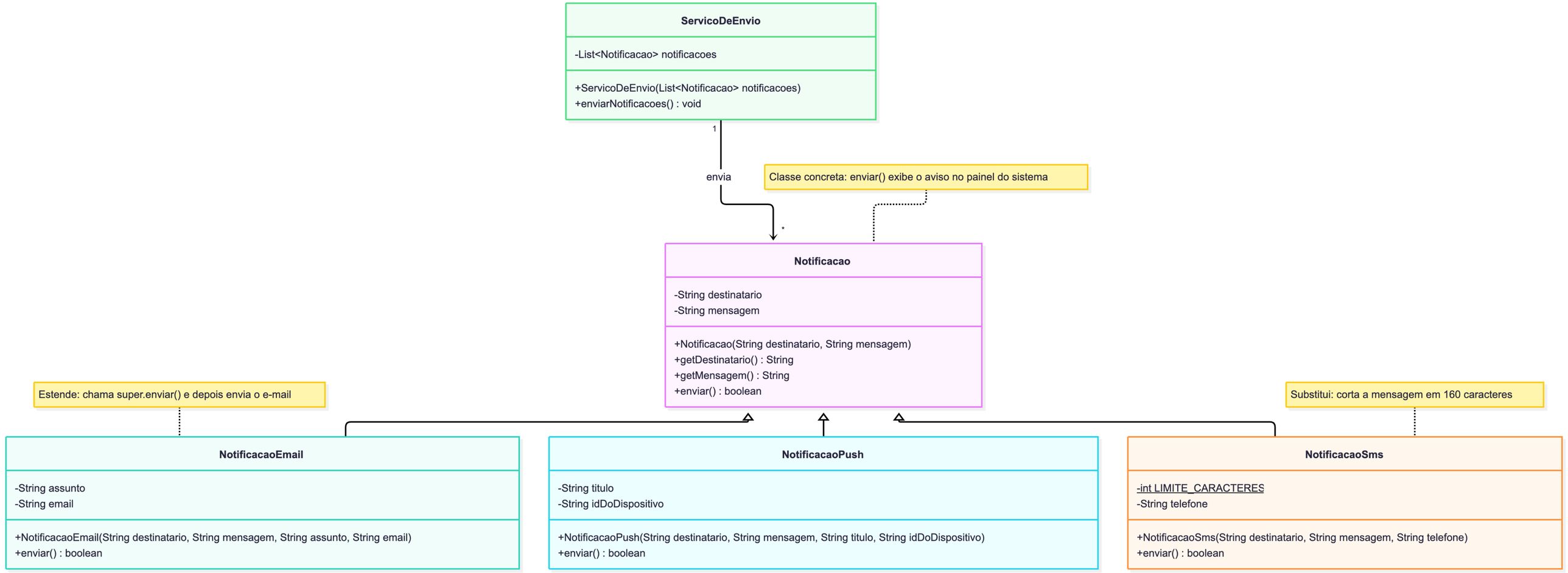
Classes abstratas

Exemplo: Notificações

- Uma loja online precisa avisar seus usuários
  - Todo aviso tem um destinatário e uma mensagem
- Atualmente, os avisos aparecem no painel do próprio sistema
- É preciso adicionar novos canais
  - E-mail: tem assunto e endereço; endereço sem @ não é enviado; o aviso também deve continuar aparecendo no painel
  - SMS: tem telefone; mensagens com mais de 160 caracteres são cortadas
  - Push: tem título e identificador do dispositivo
- O serviço que dispara os avisos deve contar quantos foram enviados e quantos falharam

Classes abstratas

Exemplo: Notificações



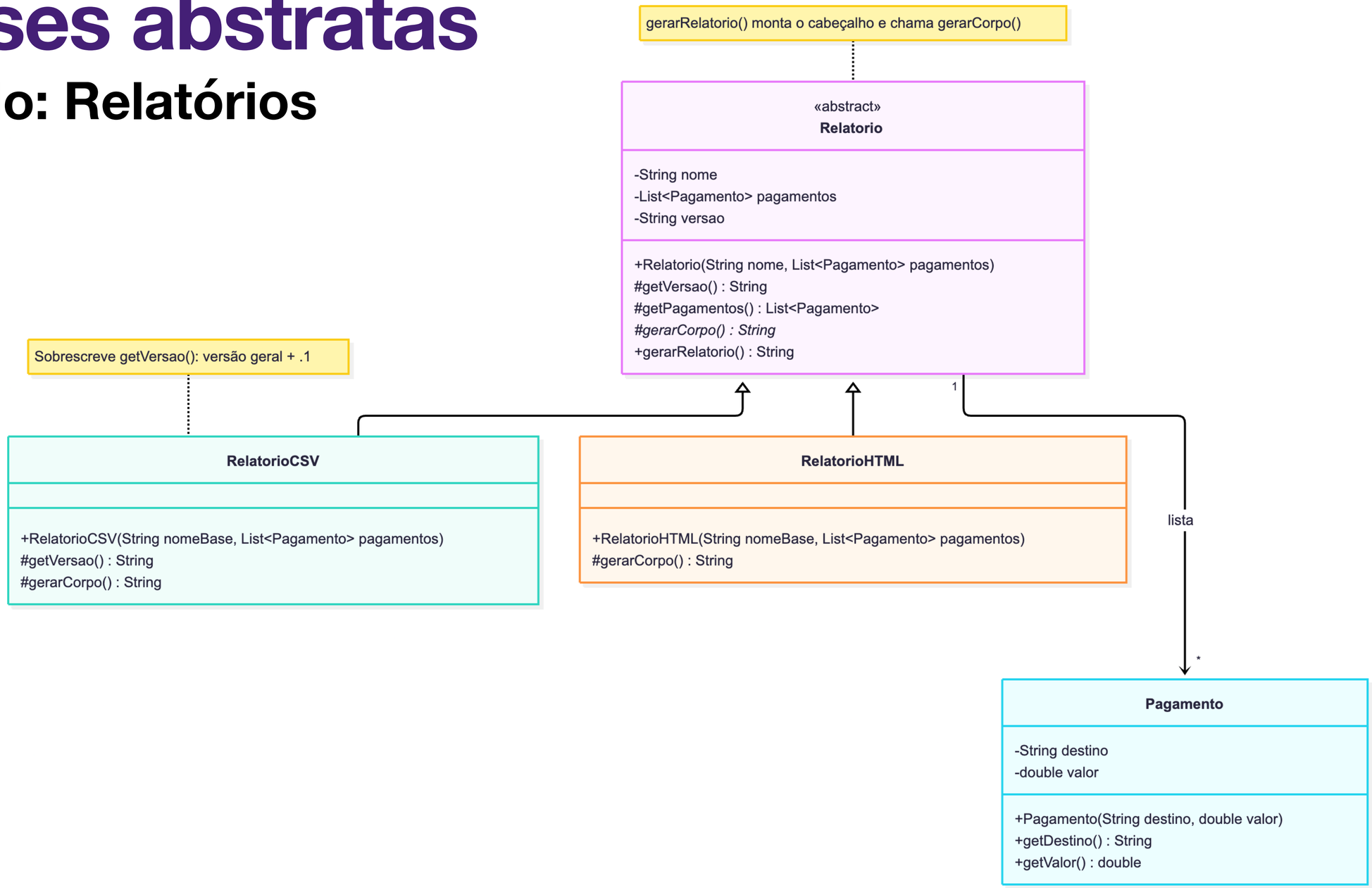
Classes abstratas

Exemplo: Relatórios

- A empresa precisa gerar relatórios dos pagamentos realizados
- Todo relatório começa com um cabeçalho: nome do arquivo e versão
- Depois vem o corpo, com a lista de pagamentos
- Formatos:
  - CSV: uma linha por pagamento, com destino e valor
    - Uma versão própria: a versão geral acrescida de .1
  - HTML: uma tabela com os pagamentos
- A extensão do arquivo depende do formato

Classes abstratas

Exemplo: Relatórios



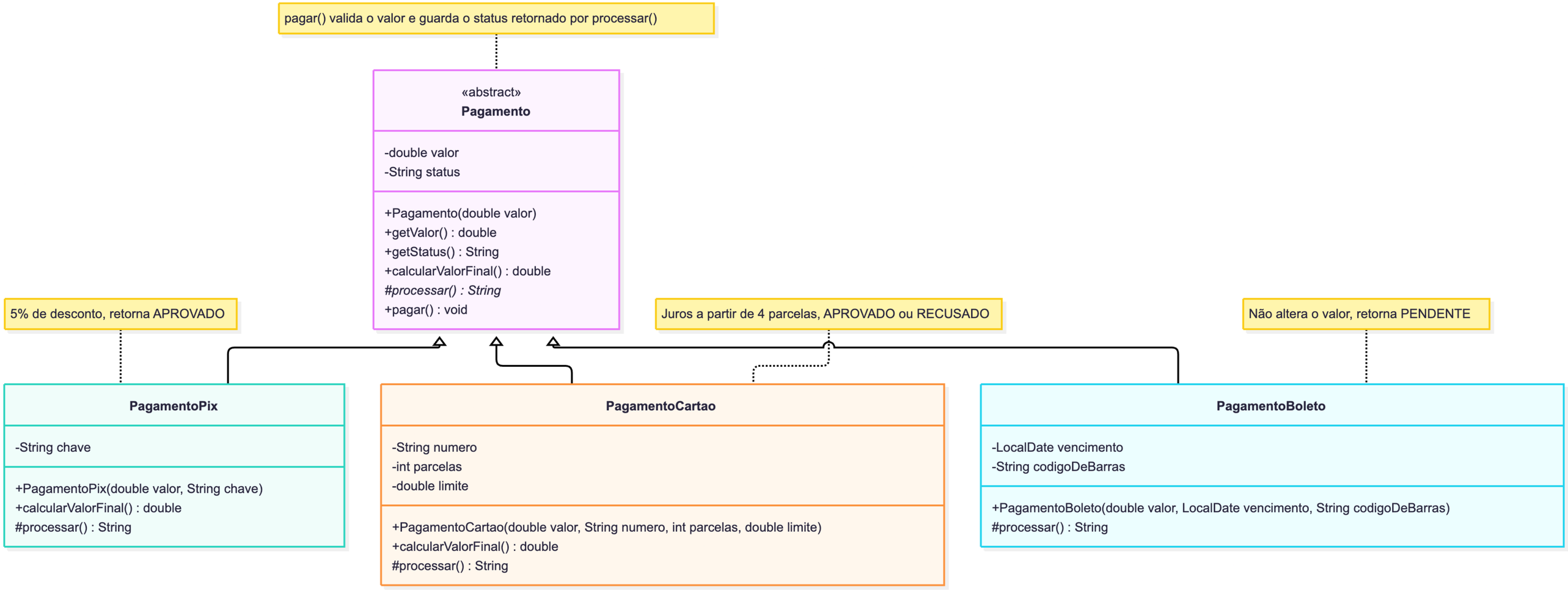
Classes abstratas

Exemplo: Pagamentos

- O checkout da loja aceita diferentes formas de pagamento
- Todo pagamento tem um valor, que precisa ser maior que zero
- Todo pagamento tem um resultado que precisa ser registrado
- Formatos:
  - Pix: tem uma chave; 5% de desconto; aprovado na hora
  - Cartão de crédito: tem número e quantidade de parcelas; a partir de 4 parcelas, acréscimo de 2% por parcela; recusado se ultrapassar o limite
- Modelem essas duas formas

Classes abstratas

Exemplo: Pagamentos



Classes abstratas

Questionamentos?

- Em qual dos problemas a superclasse pode ser instanciada?
- Em quais ela não deveria existir sozinha?
- A estrutura dos códigos é parecida. Então o que levou a decisões diferentes?
- Pergunta-chave: faz sentido existir um objeto dessa classe sozinho?

Capturando um pokemon



Capturando um pokemon



$$X = \frac{3 * HP_{max} - 2 * HP_{atual}}{3 * HP_{max}} * catch_{rate} * bonus_{ball} * bonus_{status}$$

[1, 255]

Pokédex data

National №	**147**
Type	DRAGON
Species	Dragon Pokémon
Height	1.8 m (5′11″)
Weight	3.3 kg (7.3 lbs)

Training

EV yield	1 Attack
Catch rate	45 (5.9% with PokéBall, full HP)
Base Friendship	35 (lower than normal)
Base Exp.	60
Growth Rate	Slow

Base stats

HP	41		192	286
Attack	64		119	249
Defense	45		85	207
Sp. Atk	50		94	218
Sp. Def	50		94	218
Speed	50		94	218
Total	**300**		Min	Max

Mais informações: https://www.dragonflycave.com/mechanics/gen-ii-capturing e https://bulbapedia.bulbagarden.net/wiki/Poké_Ball

Capturando um pokemon



Training

EV yield	1 Attack
Catch rate	45 (5.9% with PokéBall, full HP)

Base stats

HP	41	
Attack	64	
Defense	45	
Sp. Atk	50	
Sp. Def	50	
Speed	50	
Total	300	

[1, 255]

$$X = \frac{3 * HP_{max} - 2 * HP_{atual}}{3 * HP_{max}} * catch_{rate} * bonus_{ball} * bonus_{status}$$

$$X = \frac{3 * HP_{max} - 2 * HP_{max}}{3 * HP_{max}} * catch_{rate} * bonus_{ball} * bonus_{status}$$

$$X = \frac{HP_{max}}{3 * HP_{max}} * catch_{rate} * bonus_{ball} * bonus_{status}$$

$$X = \frac{catch_{rate} * bonus_{ball} * bonus_{status}}{3}$$

$$X = 15$$

Mais informações: https://www.dragonflycave.com/mechanics/gen-ii-capturing e https://bulbapedia.bulbagarden.net/wiki/Poké_Ball

Capturando um pokemon



Training

EV yield	1 Attack
Catch rate	45 (5.9% with PokéBall, full HP)
Base Friendship	35 (lower than normal)
Base Exp.	60
Growth Rate	Slow

$$TAXA_{modificada} = f(x)$$

 $f(x) = x$

 $f(x) = 1.5 * x$

 $f(x) = 2 * x$



 $f(x) = \begin{cases} 4 * x & \text{Se speed do Pokemon} > 100 \\ x & \text{caso contrário} \end{cases}$

 $f(x) = \begin{cases} 8 * x & \text{Se o Pokemon é do sexo oposto e da mesma espécie} \\ x & \text{caso contrário} \end{cases}$

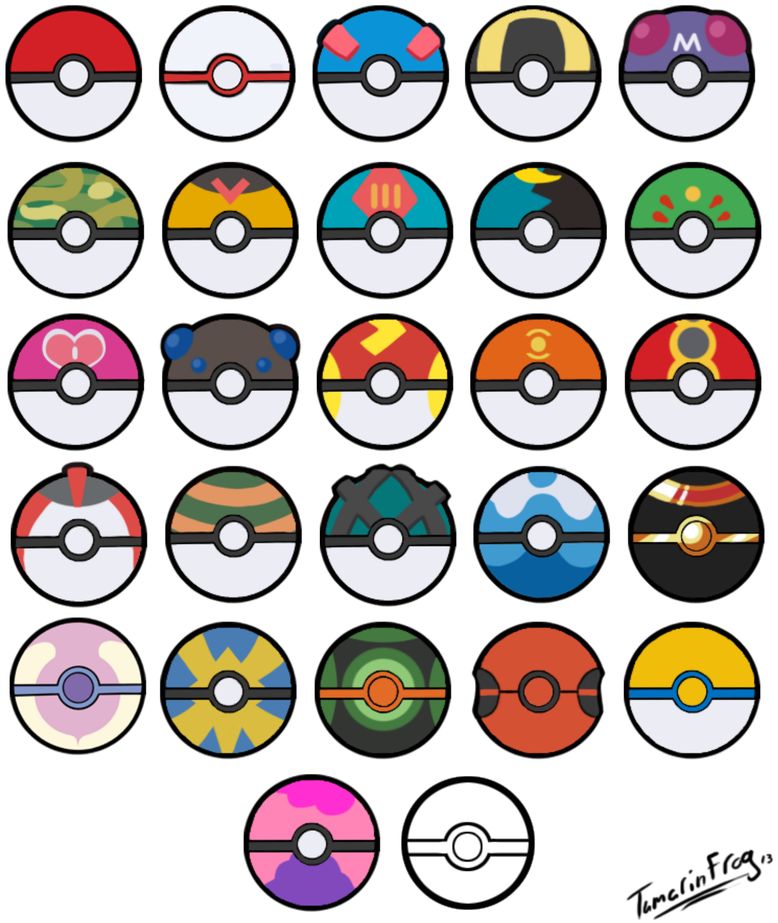
 $f(x) = \begin{cases} -20 & \text{Se peso} \leq 99 \\ x & \text{Se peso} \geq 100 \text{ e} \leq 199.9 \\ +20 & \text{Se peso} \geq 200 \text{ e} \leq 299.9 \\ +30 & \text{Se peso} \geq 300 \end{cases}$

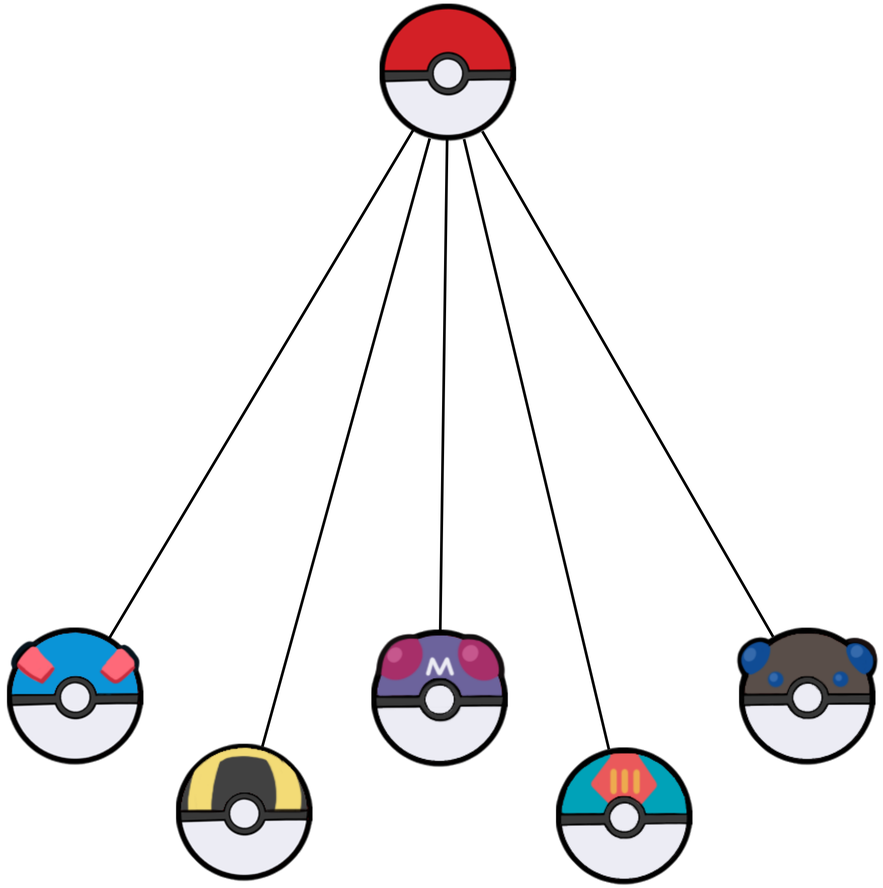


 $f(x) = \begin{cases} 5 * x & \text{Se o pokemon foi pescado} \\ x & \text{caso contrário} \end{cases}$

Capturando um pokemon

Reuso sem subtipos





Capturando um pokemon



Training

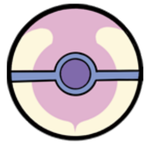
EV yield	1 Attack
Catch rate	45 (5.9% with PokéBall, full HP)

Base stats

HP	41	
Attack	64	
Defense	45	
Sp. Atk	50	
Sp. Def	50	
Speed	50	
Total	**300**	



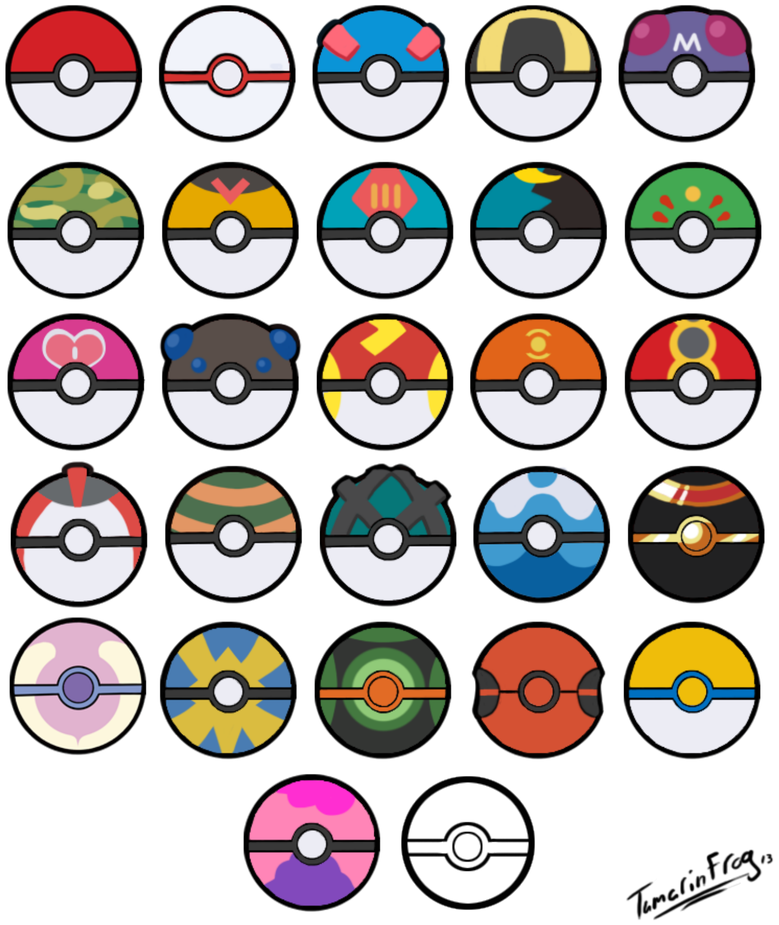
Altera os pontos de amizade do pokemon capturado para 200

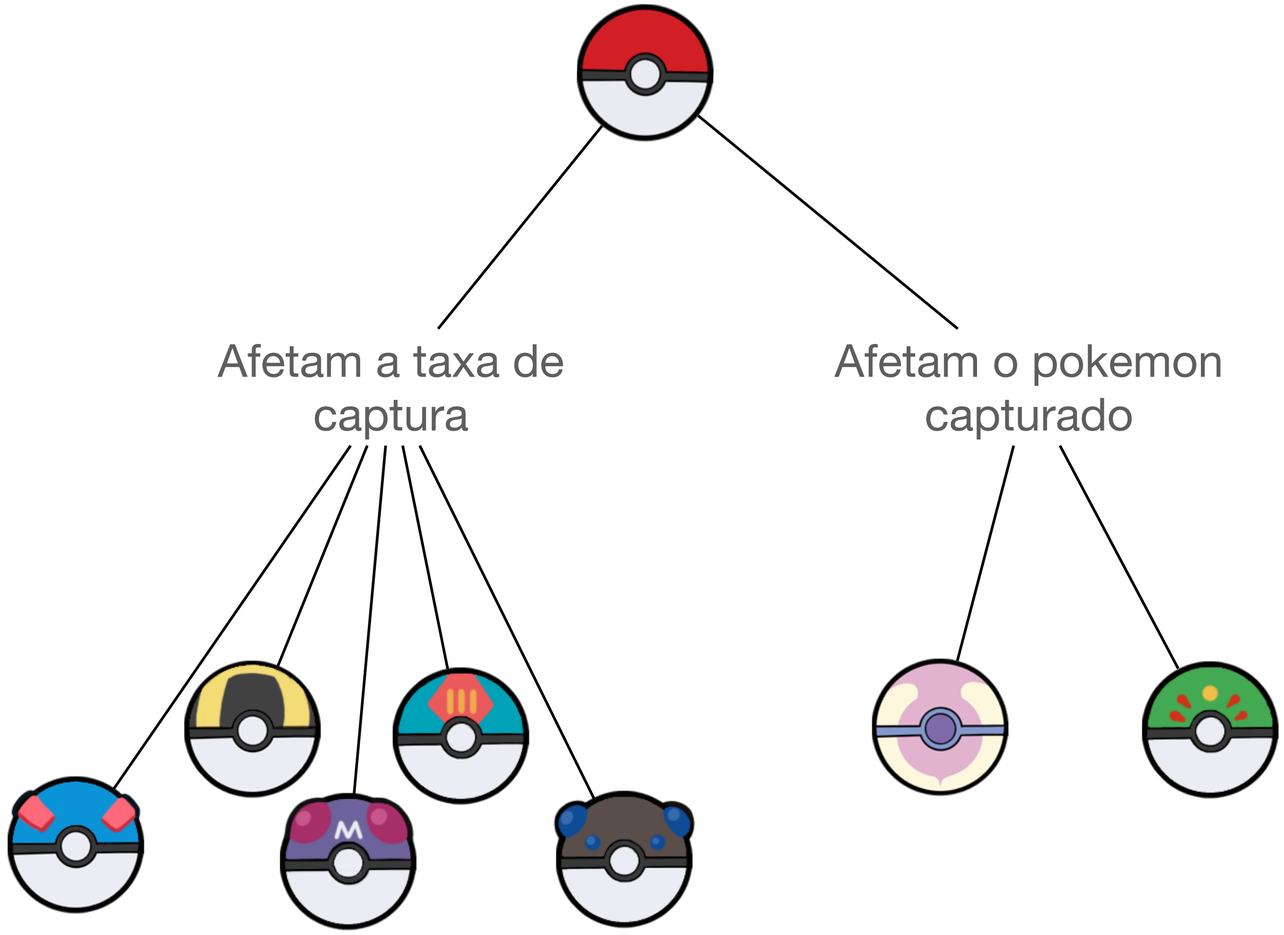


Restaura o HP, o PP e qualquer condição de status do pokemon capturado

Capturando um pokemon

Reuso com subtipos





Por hoje é só

